Table des matières

Cours 03 — Charger la RAW 1
Ce que vous allez faire 1
Avant de commencer 1
Étape 1 — Écrire `charger.py` · Ingestion 2
Étape 2 — Charger jusqu'au 11 juin · Ingestion 4
Étape 3 — Le 12, puis le 13 juin · Ingestion 6
Étape 4 — Le reste du mois · Ingestion 7
Étape 5 — Versionner · Ingestion 7
À retenir 8

Cours 03 — Charger la RAW

Cours 3/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
	✓ ici	

Ce que vous allez faire

La `RAW` est la première couche de l'entrepôt : elle garde ce qui a été livré, tel quel, sans rien interpréter. Un journal y note chaque fichier chargé par son empreinte, si bien que relancer le chargement ne charge rien deux fois. Vous écrivez `charger.py`, votre premier fichier de pipeline, puis vous faites arriver le mois jour après jour, comme si la caisse et le torréfacteur livraient chaque matin. Tout le cours est de l'ingestion : aller chercher les fichiers et les charger. Le bronze du cours 02 range les fichiers dans le lac, pour qui lit des fichiers ; la `RAW` les garde dans l'entrepôt, tels quels, pour qui écrit du SQL.

fichiers de `sources/` → `charger.py` → huit tables `raw_…` et le `journal`, dans l'entrepôt, `plateforme\entrepot\brulerie.duckdb`

Le code complet est dans `Codes_03_DE.md`, à ouvrir après avoir essayé.

Avant de commencer

© 2026 iAlexMG. Tous droits réservés.

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie`, le dossier que le cours 00 a créé :

- la première pour le Bloc-notes, Python et Git ;
- la seconde pour le notebook de DuckDB (`duckdb -ui`), où se tapent les requêtes ; elle s'ouvre à l'étape 2.

Il en faut deux cette fois : tant que le notebook est ouvert, DuckDB occupe sa fenêtre, et `charger.py` doit tourner à côté. Les deux fenêtres restent dans `Projet_Brulerie` jusqu'à la fin du cours. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
dir /b
REM Attendu : 01_de, plateforme et sources, plus ce qu'Analytics Engineering y a laissé, si vous l'avez fait.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

Ce cours part du dépôt `01_de` et du lac que le cours 02 a créés. S'ils manquent, faites le cours 02 d'abord. L'entrepôt vit à côté du lac, dans `plateforme\entrepot` : un seul fichier, `brulerie.duckdb`, que Data Engineering remplit et qu'Analytics Engineering lit, comme dans une entreprise où les deux équipes partagent un seul entrepôt. Il reste hors du dépôt, comme le lac : Git garde le code, pas les données.

Étape 1 — Écrire `charger.py` · Ingestion

Trois décisions font le chargeur :

- tout en texte : le cours 01 a montré que les types devinés trompent, et la `RAW` ne décide d'aucun type ; le bronze du cours 02, lui, les déclare ;
- le chemin du fichier sur chaque ligne (`filename = true`) : c'est le lignage, comme dans le bronze, où il a servi à garder le dernier export d'un jour ;
- le journal : une empreinte SHA-256 par fichier, calculée sur son contenu. Un fichier dont le journal connaît l'empreinte est sauté.

Le script prend un jour en argument, et ne charge que les fichiers arrivés jusqu'à ce jour-là : c'est ce qui simule un mois livré jour après jour. `lots.csv` n'a pas de date dans son nom : livré en entier, il est toujours arrivé.

Le chargeur prend huit livraisons, et non les seules que ce cours lit : les tickets et leurs lignes, le catalogue et ses modifications, les membres du programme de fidélité et leurs consentements, les lots et les courbes du torréfacteur. Les lignes, le catalogue et la fidélité servent à Analytics Engineering, qui les lira dans le même entrepôt. Charger la `RAW` en entier, pour tous ceux qui la liront, c'est le travail de l'ingénieur de données ; ces fichiers-là n'ont pas de date non plus, comme `lots.csv`.

© 2026 iAlexMG. Tous droits réservés.

Pour remplir	L'outil	Exemple
le jour d'arrivée	`fichier.stem` (le nom sans extension), `.split("_")[-1]` (son dernier morceau), `.replace("-", "")[:8]` (ses huit premiers chiffres)	`tickets_2025-06-12_202506130705` donne `20250613`, `courbe_2025-06-03` donne `20250603`, `lots` reste `lots`
comparer deux jours	deux dates `AAAAMMJJ` se comparent comme du texte	`"20250613" > "20250612"` vaut `True`
écarter `lots`, `produits` et les autres fichiers sans date	`arrive.isdigit()` est faux pour un nom sans date	
l'empreinte	`fichier.read_bytes()` rend le contenu du fichier	
le journal connaît-il le fichier	`.fetchone()` rend `None` quand la requête ne trouve rien	
un paramètre SQL	chaque `?` d'une requête prend une valeur de la liste passée à `con.execute`	`con.execute("… WHERE a = ?", [valeur])`

`CREATE TABLE … AS … LIMIT 0` crée la table avec les colonnes du premier fichier, sans aucune ligne ; l'`INSERT` qui suit la remplit, et c'est lui seul qui sert aux fichiers suivants.

1.1 Créer le fichier.

```
notepad 01_de\charger.py
```

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous ____ compris :

```
# 01_de/charger.py : charger dans la RAW ce qui est arrivé jusqu'à un jour donné
import hashlib
import sys
from pathlib import Path
import duckdb

LIVRAISONS = {"tickets": "caisse/*/tickets_*.csv", "lignes": "caisse/*/lignes_*.csv",
              "produits": "catalogue/produits.csv", "modifications": "catalogue/modifications.csv",
              "membres": "fidelite/membres.csv", "consentements": "fidelite/consentements.csv",
              "lots": "torrefaction/lots.csv", "courbes": "____"}
LECTURE = "SELECT * FROM read_csv(?, all_varchar = ____, filename = ____)"

def charger(con, jusqua):
```


© 2026 iAlexMG. Tous droits réservés.

```
    con.execute("CREATE TABLE IF NOT EXISTS journal (fichier TEXT, empreinte TEXT, charge_le TIMESTAMP)")
    for table, motif in LIVRAISONS.items():
        for fichier in sorted(Path("sources").glob(motif)):
            # 1. le jour d'arrivée, en fin de nom ; 2. sauter ce qui n'est pas arrivé, ou déjà au journal
            arrive = ____
            empreinte = hashlib.sha256(____).hexdigest()
            deja = con.execute("SELECT 1 FROM journal WHERE ____", [empreinte]).fetchone()
            if ____:
                continue
            # 3. dans raw_<table>, tout en texte, avec le chemin ; 4. noter le fichier au journal
            con.execute(f"CREATE TABLE IF NOT EXISTS raw_{table} AS {LECTURE} LIMIT 0", [str(fichier)])
            ____
            ____

if __name__ == "__main__":
    with duckdb.connect("plateforme/entrepot/brulerie.duckdb") as con:
        charger(con, sys.argv[1])
```

Enregistrer, puis fermer le Bloc-notes.

Étape 2 — Charger jusqu'au 11 juin · Ingestion

`charger.py` écrit dans l'entrepôt, et le notebook le lit. Or DuckDB ne laisse écrire dans un fichier que le programme qui est seul à l'ouvrir : tant que le notebook tient l'entrepôt, même en lecture seule, `charger.py` est refusé. Le notebook attache donc l'entrepôt pour le lire (`ATTACH … (READ_ONLY)`), et le détache (`DETACH`) avant chaque chargement.

2.1 Créer le dossier de l'entrepôt, puis lancer le chargeur sur les onze premiers jours de juin, dans la première fenêtre. DuckDB crée le fichier, mais pas le dossier qui le porte.

```
mkdir plateforme\entrepot
REM Attendu : rien ne s'affiche ; si Analytics Engineering l'a créé avant vous, « Un sous-répertoire
REM ou un fichier plateforme\entrepot existe déjà. », sans conséquence.
python 01_de\charger.py 2025-06-11
REM Attendu : rien ne s'affiche, l'invite revient.
dir /b plateforme\entrepot
REM Attendu : brulerie.duckdb ; le chargeur a créé l'entrepôt.
```

Si le chargeur s'arrête sur	La cause, et le geste
`IO Error: Cannot open file "…\plateforme\entrepot\brulerie.duckdb": Le chemin d'accès spécifié est introuvable.`	le dossier `plateforme\entrepot` manque : faire le `mkdir`, puis relancer

© 2026 iAlexMG. Tous droits réservés.

2.2 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Le notebook s'ouvre dans le navigateur. Laisser cette fenêtre ouverte : la fermer arrête le notebook. Créer un notebook (+ à côté de Notebooks) et le renommer « Cours 03 » (double-clic sur son titre, puis Entrée).

2.3 Écrire les deux premières cellules du notebook, qui resserviront à chaque chargement. La première attache l'entrepôt en lecture seule ; l'exécuter (Ctrl+Entrée).

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/brulerie.duckdb' AS entrepot (READ_ONLY);
-- Attendu : entrepot paraît sous Attached databases.
```

La seconde le détache ; l'écrire sans l'exécuter. Grâce à `IF NOT EXISTS` et à `IF EXISTS`, l'une et l'autre se relancent sans erreur, même deux fois de suite.

```
DETACH DATABASE IF EXISTS entrepot;
```

Chaque requête qui suit va dans une cellule neuve, sous ces deux-là, et nomme ses tables en entier (`entrepot.journal`) : une cellule travaille en mémoire tant qu'on ne lui nomme pas l'entrepôt.

2.4 Compter les fichiers du journal.

```
SELECT COUNT(*) AS fichiers FROM entrepot.journal;
-- Attendu : 30, soit 11 exports de tickets et 11 de lignes, 3 courbes, et les 5 fichiers sans
-- date : le journal des lots, le catalogue et ses modifications, les membres et leurs consentements.
```

2.5 Regarder ce que la `RAW` a gardé, une requête par cellule.

```
DESCRIBE entrepot.raw_tickets;
-- Attendu : 14 colonnes, toutes en VARCHAR ; la dernière, filename, porte le chemin du fichier.

SELECT COUNT(*) AS lignes FROM entrepot.raw_tickets;
-- Attendu : 575.
```

Chaque chargement qui suit se fait en trois temps. Relancer une cellule : cliquer dedans, puis Ctrl+Entrée.

Temps	Où	Quoi
1	le notebook	relancer la seconde cellule, le `DETACH` : le notebook lâche l'entrepôt
2	la première fenêtre	`python 01_de\charger.py …`

© 2026 iAlexMG. Tous droits réservés.

Temps	Où	Quoi
3	le notebook	relancer la première cellule, l'`ATTACH` : le notebook relit l'entrepôt

Un `DETACH` oublié se voit tout de suite : `charger.py` s'arrête sur `IO Error: Cannot open file "…\brulerie.duckdb": Le processus ne peut pas accéder au fichier car ce fichier est utilisé par un autre processus.`, suivi de `File is already open in …\duckdb.exe`, le programme du notebook. Rien n'est chargé : relancer le `DETACH`, puis la commande.

Étape 3 — Le 12, puis le 13 juin · Ingestion

Le 12 juin, la caisse a exporté la journée à 15 h 42 ; le second export n'arrive que le lendemain matin.

3.1 Charger le 12 juin en trois temps (`DETACH`, chargement, `ATTACH`), puis lire la dernière ligne du journal.

```
python 01_de\charger.py 2025-06-12
```

```
SELECT fichier FROM entrepot.journal ORDER BY charge_le DESC LIMIT 2;
-- Attendu : les deux fichiers de l'export de 15 h 42, lignes_2025-06-12_202506121542.csv et
-- tickets_2025-06-12_202506121542.csv.
```

3.2 Charger le 13 juin en trois temps, puis compter les tickets du 12 par fichier, et tout ce que la `RAW` porte.

```
python 01_de\charger.py 2025-06-13
```

```
SELECT PARSE_FILENAME(filename) AS fichier, COUNT(*) AS tickets
FROM entrepot.raw_tickets
WHERE CONTAINS(filename, '2025-06-12')
GROUP BY fichier
ORDER BY fichier;
-- Attendu : 58 tickets dans l'export de 15 h 42, 65 dans celui du 13 à 7 h 05.

SELECT (SELECT COUNT(*) FROM entrepot.journal) AS fichiers,
       (SELECT COUNT(*) FROM entrepot.raw_tickets) AS lignes;
-- Attendu : 37 fichiers, 752 lignes.
```

La `RAW` garde les deux exports, chacun avec son chemin : elle garde ce qui a été livré. Choisir le bon revient à qui la lit : le bronze du cours 02 l'a fait pour le lac, et Analytics Engineering le fera dans l'entrepôt.

3.3 Relancer le 13 juin en trois temps, pour prouver que relancer ne charge rien deux fois.

```
python 01_de\charger.py 2025-06-13
```

Puis relancer la dernière cellule de 3.2. Attendu : 37 fichiers, 752 lignes, comme avant la relance.

© 2026 iAlexMG. Tous droits réservés.

Chaque fichier a été relu et son empreinte recalculée, mais le journal les connaissait tous : rien n'est entré. C'est l'idempotence, et c'est ce qui permet de relancer un chargement sans crainte.

Étape 4 — Le reste du mois · Ingestion

4.1 Charger jusqu'au 30 juin en trois temps, puis compter ce que la `RAW` porte.

```
python 01_de\charger.py 2025-06-30
```

```
SELECT (SELECT COUNT(*) FROM entrepot.journal) AS fichiers,
       (SELECT COUNT(*) FROM entrepot.raw_tickets) AS lignes,
       (SELECT COUNT(DISTINCT ticket_id) FROM entrepot.raw_tickets) AS tickets;
-- Attendu : 75 fichiers, 1585 lignes, 1527 tickets.

SELECT (SELECT COUNT(*) FROM entrepot.raw_lots) AS lots,
       (SELECT COUNT(*) FROM entrepot.raw_courbes) AS lectures;
-- Attendu : 47 lots, 3718 lectures.
```

1 585 lignes pour 1 527 tickets : les 58 tickets du premier export du 12 y sont deux fois, comme au premier regard. Pour la `RAW`, c'est normal.

4.2 Lister les tables de l'entrepôt.

```
SHOW TABLES FROM entrepot;
-- Attendu : 9 tables, journal et les huit raw_…, de raw_consentements à raw_tickets ; si vous
-- avez fait Analytics Engineering avant, ses vues et ses tables s'y ajoutent.
```

Les cinq que ce cours n'a pas lues, `raw_lignes`, `raw_produits`, `raw_modifications`, `raw_membres` et `raw_consentements`, attendent Analytics Engineering, dans le même entrepôt.

4.3 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer. Le notebook « Cours 03 » reste enregistré : il sera là au prochain `duckdb -ui`.

Étape 5 — Versionner · Ingestion

5.1 Valider le chargeur, dans la première fenêtre ; l'entrepôt, hors du dépôt, n'y entre pas.

```
git -C 01_de add charger.py
git -C 01_de commit -m "Cours 03 : charger la RAW, avec son journal"
REM Attendu : 1 file changed.
```


© 2026 iAlexMG. Tous droits réservés.

```
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- La `RAW` garde ce qui est livré, tel quel : tout en texte, avec le chemin du fichier de chaque ligne.
- Le journal rend le chargement idempotent : un fichier dont l'empreinte est connue ne se recharge pas, et relancer ne double rien.
- La `RAW` garde les deux exports du 12 juin ; choisir le bon revient à qui la lit, comme le bronze du cours 02.

© 2026 iAlexMG. Tous droits réservés.